

THE UNIVERSITY OF TULSA

Project 3: Support Vector Machines & Decision Tree Ensemble Learning

CS-4103 Fundamentals of Machine Learning

In the Fundamentals of Machine Learning course, your projects require you to write and run Python code. You will implement, modify, or run one or more learning algorithms, train models with multiple datasets, test the accuracy of your models, and produce graphs and tables showing the performance of your models.

You will be given a code shell to start with. It comes as two notebooks: `Assignment 3.1` holds the Support Vector Machine part and `Assignment 3.2` holds the ensemble part. For this project you will use library implementations of every algorithm, and the code shell provides the cross-validation search and the accuracy functions. Your task is to choose and configure each model, call the provided functions with the right arguments, run the experiments, and report on the results. Your work is the notebook cells marked `TODO` and the report. You will not need to import any libraries besides those provided. Use the code shell as a guide, but feel free to restructure the code as you prefer. You may discuss algorithms with other students or chatbots, but it is up to you to write the code and the report.

Support Vector Machines

1 Overview

Support Vector Machines learn weights that define a linear decision boundary based on a collection of support vectors that describe the subset of the data closest to the decision boundary. You will use a support vector machine library to test different kernel functions on several binary classification datasets.

Read the Harvey items *SVM: Background Material* and *Support Vector Machines (SVMs)* under Week 6, and *Kernels* under Week 5.

2 Datasets

You will use three synthetic datasets and one real dataset. Each is a binary classification problem with two input features.

Linear Separable A linearly separable decision boundary.

Breast Cancer Wisconsin Tumor measurements from this UCI dataset, reduced to two standardized features, worst area and worst smoothness. Each of the 569 samples is a tumor labeled malignant (1) or benign (0). The classes overlap, so no boundary separates them perfectly.

Ellipse An ellipse-shaped decision boundary.

Spirals A double spiral-shaped boundary with strong margins between the spirals.

3 Analysis

The data is split into training and testing sets. Train and test the SVM models with scikit-learn's `SVC` class.

- Train a classifier for the Linear Separable dataset using the linear kernel. Test with a regularization value of $C = 1.0$. Plot the decision boundary.
- Train a classifier for each of the other three datasets using the polynomial kernel (degree=2) and radial basis function kernel. Test with both $C = 1.0$ and $C = 1000.0$ (Note that strength of regularization is inversely proportional to C). Keep the library's default polynomial kernel, $(\gamma\, x^\top x')^2$, which has no constant term. Plot the decision boundaries.

4 Report & Scoring

Include each decision boundary plot and the accuracy of each model on testing data. Note the effect of increasing the regularization parameter C. Compare the polynomial kernel's boundary on the Ellipse and Breast Cancer datasets, and say which kernel you would choose for each dataset.

- **4pts** - Results for the model of the Linear Separable dataset.
- **36pts** - Results for each model of the other three datasets are worth 3 points.

Ensemble Learning using Decision Trees

1 Overview

You have learned about ensemble machine learning algorithms that are made up of multiple small models. You will train several ensembles of decision trees with library implementations, choose their hyperparameters with cross-validation, and compare them with a single decision tree on two classification datasets.

2 Algorithms

You will use four learning algorithms, all from libraries.

Decision Tree Classifier You will compare the ensemble methods with an individual decision tree model. Use scikit-learn's `DecisionTreeClassifier` class. By default, use the Gini Index splitting condition, and create a decision node whenever the class of the sorted training data is not pure, and whenever the max depth has not been reached.

Random Forest Use scikit-learn's `RandomForestClassifier` class. This bagging algorithm trains each tree on a bootstrap sample of the training data. Limit the features from which the decision tree algorithm searches for each decision node to a random sample with size equal to the square root of the total number of features.

AdaBoost Use scikit-learn's `AdaBoostClassifier` class with a decision tree as the base learner. This boosting algorithm trains trees one after another and gives more weight to the training instances that earlier trees misclassified. AdaBoost is designed around weak learners, such as shallow trees. A tree of depth 1 is called a stump.

XGBoost Use the `XGBClassifier` class from the xgboost library. This gradient boosting algorithm fits each new tree to the errors of the ensemble built so far.

Read the Harvey items *Decision Trees*, *Decision Trees & Ensemble Methods: Background Reading*, and *Ensemble Learning* under Week 7.

3 Datasets

You will test the decision tree and ensemble methods across two datasets.

Handwritten Digits This is the same UCI ML handwritten digits dataset that you used on the previous projects.

Statlog (German Credit Data) This UCI dataset includes data about 1000 loan applications with 20 features each. The goal is to predict whether or not an applicant is creditworthy. The data has been formatted as a numeric dataset.

4 Analysis

The code shell splits each dataset into training and testing sets and provides a function that runs a 5-fold cross-validation search over one hyperparameter. Perform the following analysis with all four of the above algorithms on each of the two datasets.

Decision Tree Model

1. Using an individual decision tree model, use 5-fold cross-validation over the training set to determine the best tree depth. Test with max depths of 1, 5, and with no max depth.
2. Using the best tree depth found in part 1, train a new decision tree model on the full training set, then test it on the test set.

Ensemble Models

3. For each of the three ensemble model algorithms, use 5-fold cross-validation to test various max-depth values for the decision trees, using 100 trees. Test with max depths of 1, 2, and 3.
4. For each of the three ensemble model algorithms, use 5-fold cross-validation to test various tree counts. Use a max tree depth of 3 for the trees in the ensemble classifier. Test using 5, 10, 50, and 100 trees.
5. Using the best tree depth found in part 3 and the best tree counts found in part 4, train new ensemble models for all three algorithms on the full training set, then test them on the test set.

5 Report & Scoring

Provide the results of your hyperparameter searches and the accuracy of your models on the test set in your report. Compare the accuracy of the models across different algorithms.

- **28pts** - Results and analysis of each hyperparameter search are worth 2 points.
- **32pts** - Results and analysis of each model trained from the full training set when predicting test data are worth 4 points.